Lab 3: IMU Noise and Performance Characterization

Assigned: Feb. 23, 2026 Due: Sat., March 14 by 11:59pm on GitHub and Canvas

Goals and Learning Objectives

The purpose of this lab is to understand sources of noise in sensors and in the surrounding environment, and how to characterize and choose IMU sensors for different robotic applications. You will write a device driver for the IMU, collect rotation, acceleration, magnetometer, and rotational rate data, and analyze noise parameters using Allan variance plots.

After this lab, a successful student will be able to:

- Write a ROS2 device driver that communicates with the VectorNav IMU over USB serial
- Identify device parameters (e.g., angle random walk, bias stability, rate random walk) and sources of error from an Allan variance analysis of long-duration data
- Discuss sources of noise in the environment and their impact on sensor performance

Collaboration Policy for this Lab

- Each person in the team needs to write their own ROS2 device driver for the IMU.
- Each person in the team needs to collect 5 min of stationary data
- The team collects one data set of some dynamic motion
- Each person in the team gets one long-term data set for analysis.
- All data analysis should be done individually.

Hardware and Sensors: Vectornav VN-100 IMU
Sensor Emulator, see Piazza/Resources/Lab3 Serial Emulator/

Individual work: hardware setup

Standard Ubuntu is not designed for robotic applications, so for sensors with a high sampling rate, it may not behave as we want it to. There may be latency in terms of its response to the sensor. There are a set of files under /etc/udev that dictate how Ubuntu will react to certain sensors. These files instruct Linux what to do when a particular device is plugged in. More here

To make the VN-100 plug & play and to address Ubuntu USB latency issues:

1. Plug in or emulate your IMU and in the terminal: $ dmesg | tail -20
You will see your sensor attributes including product id and vendor id. These will be listed in the most recent logs after dmesg once you plug the device in.

2. Use a text editor to see the contents of /etc/udev/rules.d/50-VN-100.rules. If the values in your file differ from the values below, change them accordingly and if the file doesn't exist create it, $ sudo nano /etc/udev/rules.d/50-VN-100.rules with the contents:

```
KERNEL=="ttyUSB[0-9]*", ACTION=="add", ATTRS{idVendor}=="1d6b",
ATTRS{idProduct}=="0002", MODE="0666", GROUP="dialout"
```

4. Ubuntu's default latency of 16 ms cause issues with high-rate sensors. This problem can be solved by adding the file in /etc/udev/rules.d called 49-USB-LATENCY.rules with the contents:

```
ACTION=="add", SUBSYSTEM=="usb-serial", DRIVER=="ftdi_sio",
ATTR{latency_timer}="1"
```

5. Once the rules have been added, to get udev to recognize the rule, run the following command:

```
sudo udevadm control --reload-rules && sudo service udev restart && sudo udevadm trigger
```

6. Finally, unplug and replug the VN-100 to have it work with the new rules.
You can verify the applied settings with the below command, it should report 1 on success. (Please verify each time you collect data to be safe).

$ cat /sys/bus/usb-serial/devices/<ttyUSB0 or your device path>/latency_timer

<u>Individual work: write a ROS2 device driver for IMU and do stationary noise analysis</u>

1. Using the emulator, open a serial port with 115200 baudrate. Parse the output $VNYMR string to get accel, gyro, orientation (Euler angles: roll, pitch, yaw), and magnetometer data. Refer to the user manual for more information. *Do not use pynmea library, it has very bad checksum handling problems.*

2. Use the standard ROS sensor_msgs/Imu and sensor_msgs/MagneticField to publish the data. Your custom msg file called "IMUmsg.msg" needs to contain the following message variables:
   - ROS Header with the name of header (The frame_id will be "IMU1_Frame". You may use the system time for the Header time stamp but keep the highest precision by using both the secs and nsecs)
   - ROS sensor_msgs/Imu with the name imu
   - ROS sensor_msgs/MagneticField with the name mag_field
   - (optional) A string with any name containing the raw IMU string (this can help if you do not have the IMU and find a mistake in your driver after data collection).

3. Configure your IMU to output data at 40 Hz by writing a string to the register. This is a non-trivial step and should not be solved by just sleeping your driver to collect at 40 Hz. You will need to write to a register of the VectorNav. Before you get the actual IMU, you can check if your string is correct by writing the write register string as an option -V to the emulator and then the write register string. Confirm that your rate is correct using minicom or another serial emulator.

4. Test everything on the actual IMU.

5. Begin collecting a rosbag for the 3 accelerometers, 3 angular rate gyros, 3 axis magnetometers, and raw string. Collect at 5 – 10 minutes of data with the *instrument stationary and as far away as possible from your computer or other computers, moving objects, etc.*

6. Plot time series of each axis of data in your report. You can plot the X, Y, and Z data on the same graph. Make sure to include a legend and label your exes!
    a. Gyro, X, Y, Z
    b. Accelerometer, X, Y, Z
    c. Magnetometer, X, Y, Z
    d. Orientation (Euler angles), X, Y, Z
    e. Calculate the mean and median the orientation time series data and use a series of histograms (X, Y, Z) to plot the sensor output distribution around the median. What type of distribution(s) do you get?

<u>Group work: Action data collection (group work) and analysis (Individual)</u>

1. For one data set per team, collect roughly 5 - 10 minutes of IMU data as one teammate moves the IMU, one teammate sets up the rosbag, and one teammate records a video of the motion using their phone. This motion can be simple (moving along each axis) or complex (dance moves?), but please don't overstress the IMU cable or connector. Share the rosbag and video among the group.

2. **As a group** use the time series data from all axes to align the video time with the VectorNav data. Plot three different 5 second "clips" of interesting/relevant IMU output (doesn't need to be all sensors or axes) and include the three screengrabs near where in the video that the identified motion occurred to illustrate.

3. **Individually** study and discuss the results of this experiment in your report. Do the motions correlate well with the data?

<u>Individual work: Allan Variance analysis</u>

Five .bag files have been uploaded to Piazza, each of which contains long-duration data from the VN100.

1. Each team member should take one of the bag files and perform the following Allan Variance.
2. See The Allan Variance Tools file on Piazza/Lab3/ for information and links on performing an Allan Variance analysis. Note this is a Jupyter notebook file.

3. Use the process described on this Mathworks page to make Allan variance plots of the gyro X, Y, and Z and extract: rate random walk (*K*), angle random walk (*N*), and bias stability (*B*) parameters from your data set and plots for each axis. The following MathWorks webpage provides code that you can use to analyze your data for Allan variance.

4. Do the same Allan variance plots and analysis for the X, Y, and Z accelerometer data.

5. Compare your Allan variance results with those of your teammates.

Answer the following questions for your report:

1. What can you say about the noise in your short stationary data?
2. What was the median, mean, and distribution?
3. How do your measurements compare to the performance listed in the VN100 datasheet?
4. If the performance in your dataset was worse than the datasheet, what sources of noise do you think were present? Why?
5. How did you align the video and Vectornav action data?
6. Did the IMU data correlate well with video of the actual motion?
7. Can you make any inferences about the conditions under which your long-duration data was taken? For example, was there any periodicity to the noise indicating the presence of a machine operating nearby.
8. What do your results tell you about the conditions you should measure Allan variance under to understand the best sensor performance possible?
9. If you were trying to characterize a new sensor with unknown performance, briefly (a few sentences) explain what, where, and how you would measure to understand sources of noise in the sensor and environment, and the performance you could expect under its operating conditions.

Grading Rubric:

Driver (25%)		
	VNYMR string parsing	10
	ROS publisher node	13
	Rate changed to 40 Hz	2
Report (75%)		
	Time series plots of VectorNav parameters (stationary)	10
	Histogram plots from stationary data (stationary)	10
	Discussion (stationary)	10
	Analysis of motion + 3 time series plots/screengrab	10
	Extraction of gyro and accelerometer parameters from long-term data sets	5
	Allan Variance plots in X, Y, Z	15
	Q1 – Q9 answers	10
	Adherence to professional writing norms in engineering	5
		100

How to Submit LAB 3

1. Follow the below repo structure table along with the exact naming conventions for files and folders given below.
2. Push your local commits to (remote) GitHub server. You can verify this by visiting github.com and making sure you can see the commit there.
3. Use the Lab 3 "assignments" on Canvas to upload your report.
4. Place any MATLAB / python code you used to generate and plot noise parameters in the Analysis directory.
5. Place your individually collected bag file in the Data directory
6. Ensure you have submitted the correct files & folders by cloning a **fresh** copy of your repository in some new directory in your /home/user/ folder on linux (such as /home/user/LAB3) & running colcon build + testing the driver

Repo structure table:

EECE5554				
	LAB3			
		src		
			imu_driver	
				imu_driver (Python package with imu_driver.py file)
				launch (directory with the imu_launch.py file)
				package.xml and other files present in your original workspace
			ROS2 package for custom message	
				msg (folder) containing IMUmsg.msg
				CMakeLists.txt and package.xml
			Data	
				bag files
			Analysis	
				Python and/or MATLAB code for data analysis
				Report.pdf

Checklist For a Correct ROS2 Package Structure

- The ROS2 package name is imu_driver
- The driver is in a folder called imu_driver with the name imu_driver.py
- The message file name is IMUmsg.msg with the names of message variables & types described in the section for writing a device driver.
- You do not have a fixed USB port for the driver & it can take any USB port given by the user at run time.
- Your topic name is “imu”
  ros2 launch imu_driver imu_launch.py port:="/dev/tty*"
  where * can be any port

Please make sure your driver will run properly! By this lab, we expect that you will be able to accurately replicate a file structure and push to GitHub using the Git CLI, so your project will not earn full credit unless your driver runs under the rules specified above.